Interview Assignment: Build an Agentic Software Engineering System- URL Shortener

1. Objective

Build a working prototype that transforms a requirement into a reviewable engineering outcome using an agentic execution model. Demonstrate requirement understanding, task decomposition, multi-step execution, and output generation/validation. Focus on end-to-end SDLC automation with controlled autonomy.

2. Scenario

You will build a URL shortener service from scratch with core APIs, analytics, and reliability features.
Your task is to complete and improve it over 2-3 days using AI assistance (Copilot/Claude/etc.) while **demonstrating engineering judgment**.

3. Scope

- Greenfield scenarios (new systems/features)
- Brownfield scenarios (enhancements, refactors, bug fixes)
- Test and documentation improvements
- Well-defined and ambiguous requirements

4. Core Requirements

1) Requirement Understanding: Interpret intent, identify ambiguity, normalize into a clear engineering problem.

2) Task Decomposition: Convert high-level requirements into actionable tasks with dependencies and sequencing.

3) Codebase Reasoning (Brownfield): Identify impacted modules/services/APIs/data flows and demonstrate architectural understanding.

4) Workflow Orchestration (Critical Differentiator): Design and implement an agentic orchestration layer that coordinates the full SDLC lifecycle across requirements, architecture/design, implementation, testing, documentation, release readiness, and demonstrates non-linear, stateful execution with governance rather than simple linear task chaining. The orchestration must use an explicit dependency graph with entry/exit gates,

support sequential and parallel paths with synchronization, preserve cross-stage context and decision lineage, enforce human approval checkpoints for high-impact actions, and include bounded retries, fallback, rollback, and safe-stop controls. It must also embed policy guardrails for security, compliance, and change control, provide audit-grade observability and traceability, track reliability metrics such as success rate, retry/rollback frequency, MTTR, and end-to-end latency, and dynamically re-plan when upstream outputs change while maintaining governance and controlled agent autonomy.

5) Engineering Output Generation: Produce production-quality code, API/schema definitions, unit/integration tests, and supporting documentation with clean design and maintainability.

6) Validation and Risk Control: Identify risks/trade-offs/failure scenarios and define validation and safety guardrails.

7) Controlled Autonomy: Agents execute multi-step work; humans provide oversight, approvals, and final quality control.

8) Final Engineering Summary: Include plan/rationale, artifacts, risks/trade-offs/validation, assumptions, and limitations.

5. Deliverables

- Working prototype (runnable end-to-end)
- Architecture overview (components, orchestration model, control flow, key decisions)
- Three scenarios: greenfield, brownfield, ambiguous (each shows decomposition, orchestration, validation)
- Setup instructions
- Testing approach, limitations, and trade-offs

6. Evaluation Criteria

- Effectiveness of agentic orchestration
- Architecture/system design quality
- Depth of decomposition and execution quality
- Realism/quality of outputs
- Validation and risk management rigor
- Clarity and defensibility of decisions
- Core engineering principles: modular, testable, reliable, secure, scalable code with safe change management
- Engineering judgment

7. Expectation

Treat as production-grade engineering work. Demonstrate strong design fundamentals, lifecycle orchestration capability, output ownership, and defensible reasoning. Principle: Agents execute under defined autonomy boundaries; humans own oversight, approvals, and final quality.